Back to assignments

ASSIGNMENT 2

Burst-parallel ML with AWS Lambda

Due Fri Jun 5, 11:59 PM ET

FAQs

- **PLEASE DO NOT PROCRASTINATE!!** If you wait till the last minute to work on your assignment and ask for help, no one will be able to help you. **A2 is more challenging than A0/A1**, even with the help of AI coding tools (OpenRouter models): some steps, especially deploying AWS resources such as S3 buckets and Lambda functions, may take longer (and more tokens) than you expect. So, start working on it **EARLY**.
- This is a group assignment. Each group should submit one solution to Canvas.
- Use **OpenCode** as your default AI coding CLI. If you already have access to Anthropic Claude Code, OpenAI Codex CLI, or another AI coding CLI, you may use that instead. Your transcript should make clear what tool you used and how you used it.

> **Important Note 1: keep your Lambda concurrency small.** Do not test with a large dataset that may invoke more than 20 concurrent Lambda workers. To be safe in AWS Academy Learner Lab, use a small dataset that we provide and configure your system with a worker concurrency limit of **5**. If you exceed the service or concurrency limits of your Learner Lab account, **your account may be deactivated**.

> **Important Note 2: back up your work frequently.** Keep a copy of your source code, deployment scripts, notes, and transcript in a safe place. A private GitHub repository is recommended. **If your AWS Academy account is deactivated, your resources and data may be removed, and you may no longer be able to access the account.**

> **AWS Academy credentials expire when your Learner Lab session changes.** When you start a new Learner Lab session, the old AWS CLI credentials copied to your EC2 instance may stop working. If deployment commands fail with credential errors, go back to **AWS Details** on the Learner Lab dashboard, click the **AWS CLI Show** button, copy the latest credentials, and paste them into your EC2 instance's `~/.aws/credentials` again.

Overview

This assignment is designed to support your in-class understanding of serverless computing, burst-parallel execution, object storage, and machine learning (ML) inference as a data systems workload. You will use an AI coding agent to build a small serverless sentiment-analysis framework on **AWS Lambda** and **S3**.

Similar to the in-class demo of the Lambda-based Raid Boss game design, your system should expose a static website hosted from S3. A user should be able to open this website in a browser, upload an IMDb review dataset, start sentiment analysis, and watch results appear as the Lambda worker fleet processes the uploaded data.

The ML task is binary sentiment classification on IMDb-style movie reviews. You should use a small subset of the Hugging Face IMDb dataset (download the CSV file **here** -- note it only contains 1,000 rows), and a **DistilBERT model** fine-tuned for IMDb sentiment classification: `dfurman/distilbert-base-uncased-imdb`. The full IMDb dataset can be downloaded from `stanfordnlp/imdb`. If you do test this full dataset, there is a risk your account may hit the service limit of Academy Learner Lab. If so, your account may be deactivated.

Your system should automatically partition uploaded data into shards and invoke multiple Lambda workers in parallel. Each worker should process one shard, write its output to S3, and make its progress visible to the website. You will use the system to study how shard size and Lambda concurrency affect end-to-end processing time.

Part 0: Environment setup

You will complete this assignment in your existing AWS Academy Learner Lab account.

You should use your AI coding CLI to install and configure the software tools needed for deployment. For this assignment, I recommend the following models: `Qwen 3.6 Plus` for implementation, and `GLM 5` for deploying AWS S3 and Lambda resources. But you can use one model throughout. Baed on my experience, Qwen may get stuck using AWS tools for deployment.

A reasonable implementation will use:

- AWS CLI;
- AWS SAM CLI or an equivalent reproducible deployment tool;
- Docker, for building the Lambda container image (you will need various Python dependencies including PyTorch-CPU, that's why you need to build a Lambda container image);
- Hugging Face `transformers` and PyTorch inside the Lambda worker image.

The exact deployment mechanism is up to your group. However, you must be able to explain and reproduce it. For example, you may use AWS SAM plus AWS CLI, Terraform, CDK, or boto3 deployment scripts. Manual clicking in the AWS Console is not recommended for creating the core resources, because it is harder to reproduce and can be challenging for new learners to configure it right.

When using an AI coding CLI to create or deploy **Lambda functions** in AWS Academy Learner Lab, explicitly tell the AI tool to use the existing AWS Academy IAM role named `LabRole`. Do not ask the AI tool to create a new IAM role for Lambda. AWS Academy accounts often restrict IAM role creation, and using a new role can cause deployment failures. Your coordinator Lambda and worker Lambda should both be configured to use `LabRole`, either by referencing the role ARN in your deployment template or by looking it up during deployment.

For example, include a constraint like this in your prompt:

```
This project runs in AWS Academy Learner Lab. Use the existing IAM role named
LabRole for all Lambda functions. Do not create new IAM roles. Do not create a
custom Lambda execution role.
```

AWS credentials on your EC2 instance

Since you are using an EC2 instance inside AWS Academy Learner Lab as your development machine, configure the AWS CLI credentials on that instance before running deployment commands.

To find your temporary AWS credentials in AWS Academy Learner Lab:

1. Open **Modules** in AWS Academy web UI.
2. Click **AWS Details**.
3. On the **Learner Lab** dashboard, click the **AWS CLI Show** button.
4. Copy the displayed credential block. It should look similar to:

```
[default]
aws_access_key_id=...
aws_secret_access_key=...
aws_session_token=...
```

On your EC2 instance, create the AWS configuration directory and credentials file:

```
mkdir -p ~/.aws
nano ~/.aws/credentials
```

Paste the credential block into `~/.aws/credentials`, save the file, and exit the editor. You can also create the directory first with `mkdir .aws` if you are already in your home directory.

These credentials are temporary. If your Learner Lab session expires or your AWS CLI commands start failing with credential errors, return to AWS Academy, copy the latest AWS CLI credentials, and update `~/.aws/credentials` again.

Part 1: System architecture

In this assignment, you will use AI to build a static website plus a two-Lambda backend. Your design should include the following components:

- **Frontend static website.** The webpage source files should be organized locally under a `frontend/` directory. At minimum, your source tree should include:

```
frontend/index.html
frontend/styles.css
frontend/app.js
```

The `config.json` file should be generated during deployment. The browser JavaScript should read `config.json` to learn the backend Lambda Function URL and any public configuration it needs.

- **Frontend S3 bucket.** Your deployment code should create an S3 bucket for the static website if it does not already exist. During deployment, upload the frontend files to the **root** of this S3 website

bucket, together with the generated `config.json`. The deployed S3 bucket should contain objects such as:

```
s3://your-frontend-bucket/index.html
s3://your-frontend-bucket/styles.css
s3://your-frontend-bucket/app.js
s3://your-frontend-bucket/config.json
```

Do not require users to open the website through a nested path such as `/frontend/index.html`. The final website should be directly accessible from the S3 static website endpoint.

The bucket should enable static website hosting:

```
WebsiteConfiguration:
  IndexDocument: index.html
  ErrorDocument: index.html
```

The bucket should have a public-read policy for the website objects so that you can open the page in a browser. The final website URL should be an accessible S3 static website URL.

- **Data S3 bucket.** Your deployment code should also create the data bucket if it does not already exist. Users should not manually create this bucket through the AWS Console. Uploaded input files, generated shards, worker outputs, and job metadata should be stored under this bucket.
- **Coordinator Lambda.** This Lambda function should expose a Lambda Function URL. The website calls this URL to request upload URLs, start jobs, poll job status, and fetch result summaries. The coordinator Lambda should handle S3 metadata updates and invoke worker Lambdas. The coordinator should enforce a worker concurrency limit. Use **5 concurrent worker Lambdas** as the recommended safe default for AWS Academy Learner Lab, and do not configure experiments that may invoke more than 10-20 concurrent worker Lambdas. Progress tracking can be simple: for example, the coordinator may track the total number of sentiment-analysis worker Lambdas for a job and how many of those workers have completed. The frontend should allow up to 20 seconds for coordinator requests before treating them as failed, because the first request may include Lambda cold-start latency (we discussed cold start issues in class). In AWS Academy Learner Lab, configure this Lambda function to use the existing `LabRole` IAM role.
- **Worker Lambda.** This Lambda function should run DistilBERT inference. Each worker invocation processes one shard from S3 and writes one output file back to S3. In AWS Academy Learner Lab, configure this Lambda function to use the existing `LabRole` IAM role.

Your AI coding CLI should help you implement this architecture. You are responsible for inspecting, testing, and understanding the generated code.

Part 2: Input data and sharding

Your system should support IMDb-style review data in CSV or Parquet format. The input data should contain the following column:

```
text
```

You can download a small test dataset with ground-truth labels **here**. The use of the test dataset is optional, only for validation purposes. If ground-truth labels are available, the input data may also

include the following optional column:

```
label
```

For example, a valid labeled CSV file may contain:

```
text,label
```

A valid unlabeled CSV file may contain:

```
text
```

The provided dataset is CSV. The original IMDb dataset is Parquet: Parquet input should contain equivalent columns. An `id` column is not required. You may optionally add an index or review id in their own implementation for debugging, but the assignment does not require one.

The IMDb labels are:

```
0 = negative
1 = positive
```

Your frontend should let a user upload a CSV (or Parquet) file. The recommended browser workflow is:

1. The browser (your webpage) calls the coordinator Lambda Function URL to request a presigned S3 upload URL.
2. The browser uploads the file directly to S3.
3. The browser calls the coordinator Lambda Function URL to start processing.
4. The coordinator partitions the uploaded file into shards.
5. The coordinator invokes one worker Lambda per shard, subject to the configured concurrency limit.

For safety, your implementation should enforce a small worker concurrency limit. The recommended limit is **5 concurrent worker Lambdas**. **Again**, do not design your experiments to launch more than 20 concurrent Lambda workers. Large inputs or very small shard sizes can accidentally create too many simultaneous worker invocations, which may exceed AWS Academy Learner Lab service limits and may cause your account to be deactivated.

Your website should **provide a configuration dropdown** that lets the user choose the shard size before starting sentiment analysis. See **Part 4: Website and live progress log** for more details. The selected shard size should control how the uploaded dataset is partitioned. In your experiments, test shard sizes between:

```
100 to 500 reviews per shard
```

Small shards increase parallelism but create more Lambda invocations. Large shards reduce invocation overhead but make each Lambda run longer. Your report should discuss this tradeoff.

Part 3: Lambda-based sentiment analysis

In this part, implement the worker Lambda.

The worker Lambda should:

1. Receive a job id, shard id, input S3 key, and output S3 prefix.
2. Download exactly one shard from S3.
3. Load the DistilBERT IMDb sentiment model:

```
dfurman/distilbert-base-uncased-imdb
```

4. Run inference over the review text in batches.
5. Write one output file for that shard to S3.
6. Update job progress metadata so that the website can display progress. At minimum, progress tracking may be as simple as recording that this shard's sentiment-analysis worker has completed.

The output for each review should include at least:

```
input_text or input_text_prefix
predicted_label
predicted_label_name
score
```

A row index or review id may be included if it is useful for debugging, but it is not required.

If the uploaded dataset includes ground-truth labels, also include:

```
ground_truth_label
correct or incorrect
```

You have two acceptable options for making the DistilBERT model available to the worker Lambda. Your `notes.txt` should clearly state which option you used and what tradeoff you observed.

Option A: Bake the model into the worker Lambda container image. During the Docker build, download `dfurman/distilbert-base-uncased-imdb` and copy the model files into the image. The worker then loads the model from the local container filesystem. This usually gives more predictable execution and can reduce cold start overhead caused by runtime model download, but it makes the container image larger.

Option B: Download the model at runtime. The worker Lambda downloads the model from Hugging Face when it starts. This keeps the container image smaller and may be simpler to implement, but the first invocation can be much slower and may fail if network access or download time becomes a bottleneck. If you choose this option, cache the model under `/tmp` during the same Lambda execution environment when possible.

In either option, the worker should use CPU-only dependencies. Do not install CUDA-enabled PyTorch, CUDA runtime libraries, or NVIDIA packages, because AWS Lambda does not provide GPUs for this assignment.

Required Lambda settings are:

```
Coordinator Lambda:
Memory: 2048 MB
Timeout: 600 seconds

Worker Lambda:
```

```
Memory: 2048 MB
Timeout: 600 seconds
Ephemeral storage: 2048 MB
Batch size: 4 or 8
Tokenizer max_length: 256
```

Configure both Lambda functions with 2048 MB memory and a 600-second timeout. This gives the coordinator enough time to shard larger input files and gives the worker enough time to load the CPU-only DistilBERT model and process a shard.

You may tune the worker batch size and report what you observed in `notes.txt`. Do not assume that more memory is always better. In Lambda, increasing memory also increases the CPU resources assigned to the function, so memory size can be treated as a performance-tuning parameter in optional experiments.

Part 4: Website and live progress log

In this part, implement the browser interface.

Your static website should include:

- a file upload control;
- a button to start sentiment analysis;
- a status area showing the current job id and progress, such as `completed_workers / total_workers`;
- a timing area showing coordinator Lambda request/execution time and worker Lambda execution time;
- a log table that appends results as shards complete, including the execution time of each completed worker/shard;
- a final summary showing total reviews processed, total elapsed time, coordinator Lambda timing, and worker Lambda timing statistics.

For static website simplicity, the frontend should poll a status endpoint every few seconds. The coordinator Lambda should return enough information for the frontend to display job progress and append result rows as worker outputs appear.

For this assignment, use S3 object-based progress tracking. Each worker should write its output file to S3 when it finishes. The coordinator should compute progress by counting completed worker outputs, such as `completed_workers / total_workers`. This can be implemented by counting output files or separate completion marker objects in S3. Do not implement progress by having multiple workers concurrently update the same S3 metadata file, because that can create race conditions.

The website dashboard should also report timing information. At minimum, it should display how long coordinator Lambda requests take, such as upload URL creation, job start/sharding, and status polling. For worker Lambdas, each worker should record its own start time, end time, and execution time for its assigned shard. The coordinator should include this timing information in the job status response so the dashboard can show per-shard worker execution time and simple summary statistics such as average, minimum, and maximum worker time.

There are no strict requirements on the exact timing metrics you report in the web UI. At minimum, your dashboard should report the end-to-end execution time of each Lambda function invocation, including coordinator tasks and worker sentiment-analysis tasks.

When using your AI coding CLI to implement the frontend, ask it to handle slow coordinator Lambda responses gracefully. A reasonable choice is to use a 20-second client-side timeout for HTTP requests to the coordinator Lambda Function URL. This avoids reporting failure too early when the first request is slow because of Lambda cold start. If a request exceeds this timeout, the website should show a clear message and allow the user to retry or continue polling.

The website does not need authentication. However, do not upload sensitive data. This assignment is only intended for public or course-provided datasets.

Part 5: Experiments

Use your system to process IMDb review data and measure performance.

Keep the experiments small enough for AWS Academy Learner Lab. Use a worker concurrency limit of **5** for normal testing. Do not use a dataset or shard configuration that may launch more than 20 concurrent worker Lambdas. Staying within this limit is part of the assignment requirement.

You should run at least three experiments:

- **Experiment 1.** Process a small input with one shard. This verifies end-to-end correctness.
- **Experiment 2.** Process the provided input CSV using multiple shard sizes selected from the range 100-500 reviews per shard. For example, if the provided training dataset contains 1,000 reviews, then using a shard size of 100 reviews will create 10 shards and launch 10 worker Lambda invocations in total. However, because your system should limit concurrency to at most 5 worker Lambda executions at a time, only up to 5 workers should run concurrently. Record the total elapsed time, coordinator Lambda execution time, and per-shard worker Lambda processing time.

For each experiment, save the timing results and a short explanation of what you learned. You should use these findings in your final notes.

Tasks

This assignment features four tasks as follows:

- **Task 1.** Implement the static website. The website should upload input data to S3, start sentiment analysis, poll for status, and display a live log of processed review entries. Configure frontend requests to the coordinator Lambda with a 20-second client-side timeout to tolerate cold starts. The dashboard should also display coordinator Lambda timing and worker Lambda execution time for completed shards.
- **Task 2.** Implement the coordinator Lambda. It should manage upload metadata, partition input files into shards of 100-500 reviews, invoke worker Lambdas subject to a safe concurrency limit, and expose status and timing information to the website. Use 5 concurrent worker Lambdas as the recommended default, and do not allow normal experiments to exceed 20 concurrent workers.
- **Task 3.** Implement the worker Lambda. It should run DistilBERT sentiment analysis on one shard, measure its own execution time, and save one output file to S3.
- **Task 4.** Run experiments and prepare your notes. Compare different shard sizes and report your observations about Lambda startup overhead, coordinator Lambda time, per-shard worker Lambda runtime, concurrency, and total job completion time.

> **NOTE:** This is an open-ended development assignment. With the help of AI coding tools, different groups may make different design choices, and there is no single required implementation that is considered the only correct solution. Your design is acceptable as long as the final prototype works end to end: the user can upload a CSV file through the website, the coordinator Lambda can shard the input data and invoke worker Lambdas, each worker Lambda can process one shard and save its output to S3, and the dashboard shows the requested information, including progress, result logs with predictions, and timing information.

Hint: How to use your AI coding CLI effectively

You may give the AI coding CLI the entire assignment Markdown file as the initial prompt. However, do not simply ask it to "build the whole system" immediately. This assignment has many moving parts, including S3 static website hosting, presigned uploads, Lambda Function URLs, Lambda container images, S3-based progress tracking, and frontend dashboard updates. If the AI makes a wrong assumption early, it may generate a system that is difficult to debug later.

A good workflow is to first ask the AI to read the full assignment spec and **interrogate** the requirements before writing code. For example, you can ask:

> Read the full assignment specification carefully. Before writing code, interrogate the spec: list the required components, identify unclear or risky parts, state your assumptions, and propose a step-by-step implementation plan. Do not generate code yet.

After reviewing the AI's plan, ask it to implement the system. There are generally two ways of building the entire system:

1. **Incremental implementation:** Ask the AI to build and test one component at a time, such as deployment scripts, coordinator Lambda, worker Lambda, and frontend.
2. **One-shot implementation:** Ask the AI to generate the full prototype at once, then debug and refine it.

Both approaches can work because the scope of A2 is small enough for a capable AI coding tool to generate most of the system in one pass. However, AI-generated code may still contain glitches, especially around AWS deployment, IAM roles, Lambda container images, CORS, S3 permissions, and frontend/backend integration.

If one model gets stuck on a small deployment issue for too long, switch to a different model rather than repeatedly asking the same model to fix the same problem. For example, if Qwen spends a long time and several dollars trying to debug a Lambda deployment issue without making progress, you may switch to another model, such as GLM, and ask it to diagnose and fix the issue based on the current code, error messages, and deployment logs.

You should also explicitly remind the AI of the assignment-specific constraints. For example:

> Important constraints: use the AWS Academy `LabRole`; use CPU-only Lambda dependencies; do not install CUDA, NVIDIA packages, torchvision, or torchaudio; use S3 object-based progress tracking; limit worker Lambda concurrency to 5 by default; do not exceed 20 concurrent workers; use 2048 MB memory and 600-second timeout for both Lambdas; configure frontend requests to the coordinator Lambda with a 20-second client-side timeout.

You are responsible for inspecting, testing, and understanding the generated code. AI-generated code is allowed, but you should not treat it as automatically correct. Use the transcript to document how you prompted the AI, what design choices it made, what errors occurred, and how you fixed them.

Deliverables

You should submit a `tar.gz` file to Canvas, which follows the naming convention of `LastName_FirstName_ComputingID_A2.tar.gz`. The submitted file should include:

- the source-code files containing the code of each task. Each file should be meaningfully named so that the teaching staff can know which file is for which task;
- the exported OpenCode or AI coding CLI session transcript markdown file that records your conversations with AI, including model responses, tool details, and assistant metadata;
- a screenshot or screen recording of your S3-hosted website processing uploaded IMDb data and displaying the live result log;
- a `notes.txt` file describing your design, how to deploy and run your system, the deployment mechanism you used, the model and dataset used, the shard sizes you tested, timing results, including coordinator Lambda time and worker Lambda time, and any findings from your experiments. Importantly, the `notes.txt` file **should also include the publicly accessible URL of your frontend website so that the teaching staff can view and test it**. See how an example URL may look like: `http://sentiment-a2-frontend-190705460846.s3-website-us-east-1.amazonaws.com/`